

UNIDAD 1. FUNDAMENTOS DE PL/SQL

1. CARACTERÍSTICAS PRINCIPALES DE PL/SQL

PL/SQL (**Procedural Language / Structured Query Language**) es una extensión de SQL que incorpora funcionalidades típicas de los **lenguajes de programación**, como estructuras de control, variables y modularidad.

- Extensión de SQL: Permite **incluir sentencias de consulta y manipulación** de datos (DML: SELECT, INSERT, UPDATE, DELETE) en unidades de código procedural. No se permiten instrucciones DDL ni DCL.
- Ejecución en Servidor: Se ejecuta en el lado del servidor de la base de datos (BD). Los procedimientos y funciones se almacenan en la BD.
- Comentarios: Comienzan con – o se colocan entre /* y */.
- Salida por Pantalla (Depuración): No tiene instrucciones de entrada/salida por teclado o pantalla. Para mostrar información (típicamente en depuración), se usa el método DBMS_OUTPUT.PUT_LINE de un paquete predefinido de Oracle.

2. ESTRUCTURA DE UN BLOQUE PL/SQL

El código PL/SQL se organiza en bloques que tienen tres secciones principales, siendo la sección ejecutable (BEGIN) y el final (END;) **obligatorias**.

SECCIÓN	CLÁUSULA	CARÁCTER	DESCRIPCIÓN
Declaración	DECLARE	Opcional	Se declaran variables, cursores y excepciones definidas por el usuario.
Ejecutable	BEGIN	Obligatorio	Contiene las sentencias SQL y las sentencias de control PL/SQL (la lógica del programa).
Manejo de Errores	EXCEPTION	Opcional	Contiene las acciones a realizar cuando se producen errores (excepciones).
Fin del Bloque	END;	Obligatorio	Finaliza el bloque.

Ejemplo de Estructura Básica

DECLARE

```
v_usuario VARCHAR2(10);
```

```
v_fecha DATE;
```

BEGIN

```
-- Sentencias SQL
```

```
SELECT user_id, fecha
```

```
INTO v_usuario, v_fecha
```

```
FROM tabla;
```

EXCEPTION

```
WHEN nombre_excepcion THEN
```

```
-- Acciones para manejar el error
```

```
NULL;
```

```
END;
```

```
/
```

3. DECLARACIÓN Y ASIGNACIÓN DE VARIABLES

3.1. Declaración

La sintaxis para declarar una variable es →

Identificador [CONSTANT] tipo_dato [NOT NULL] [:= | DEFAULT expresion];

- Ejemplo: *v_deptno* **NUMBER(2) NOT NULL := 10;**
- Constantes: Se usa la palabra clave **CONSTANT**.

3.2. Atributos %TYPE y %ROWTYPE

Estos atributos ayudan a mantener el código más robusto, ya que la variable hereda el tipo de dato de otra variable o de la estructura de la BD.

- **%TYPE**: Se usa para declarar una variable basada en otra variable previamente declarada o en una columna de la BD.
 - Ejemplo (Columna): *v_incre_sal emp.sal%TYPE := 2000;*
- **%ROWTYPE**: Se usa para declarar una variable de tipo compuesto (un registro), que puede almacenar una fila completa de una tabla o de un cursor.

3.3. Asignación

Se usa el operador de asignación **:=**.

- Directa: **identificador := expresión;**
- Desde Consulta: Se usa la cláusula **INTO** en una sentencia **SELECT** que solo devuelva una fila.

```
SELECT sal * 0.10 INTO v_comision  
FROM emp  
WHERE empno = 7082;
```

4. SENTENCIAS SQL EN PL/SQL

Las sentencias DML (**SELECT**, **INSERT**, **UPDATE**, **DELETE**) se pueden integrar en el bloque ejecutable de PL/SQL.

4.1. Recuperar Datos (SELECT ... INTO)

La cláusula **INTO** es obligatoria para almacenar el resultado de la consulta en variables o en un registro.

- Importante: La consulta debe devolver exactamente una fila. Si devuelve ninguna o más de una, se produce una excepción (**NO_DATA_FOUND** o **TOO_MANY_ROWS**).

4.2. Inserción, Actualización y Borrado de Datos (INSERT, UPDATE, DELETE)

Estas sentencias se utilizan directamente en la sección **BEGIN**.

- **INSERT** → Ejemplo:

```
INSERT INTO emp(empno, ename, job, deptno)  
VALUES(empno_sequence.nextval, 'HARDING', 'CLERK', 10);
```

- **UPDATE** → Ejemplo:

```
UPDATE emp  
SET sal = sal + v_incre_sal  
WHERE job = 'ANALYST';
```

- **DELETE** → Ejemplo:

```
DELETE FROM emp
WHERE deptno = v_deptno;
```

5. ESTRUCTURAS DE CONTROL

PL/SQL incluye estructuras condicionales y repetitivas para controlar el flujo de ejecución.

5.1. Condicionales (IF y CASE)

ESTRUCTURA	SINTAXIS CLAVE	DESCRIPCIÓN
IF	IF condicion THEN ... [ELSIF condicion THEN ...] [ELSE ...] END IF;	Permite ejecutar un bloque de instrucciones si se cumple una condición.
CASE	CASE [expresion] WHEN [condicion/valor] THEN ... [ELSE ...] END CASE;	Similar al switch de C, evalúa cada condición/valor hasta encontrar una que se cumpla.

5.2. Repetitivas (Bucles)

Los bucles repiten un grupo de sentencias varias veces.

BUCLE	USO PRINCIPAL	SINTAXIS CLAVE
WHILE	Repeticiones basadas en una condición.	WHILE condición LOOP instrucciones; END LOOP;
FOR	Repeticiones basadas en un contador (número conocido de vueltas).	FOR indice IN [REVERSE] valor_inicial .. valor_final LOOP instrucciones; END LOOP;
Básico (LOOP)	Acciones repetitivas sin condiciones globales (se necesita un EXIT explícito).	LOOP instrucciones; END LOOP;

- Nota del bucle FOR: El índice (indice) se declara implícitamente y solo está definido dentro del bucle.

6. PROGRAMACIÓN MODULAR (SUBPROGRAMAS)

PL/SQL permite cuatro tipos de bloques de código, siendo los procedimientos y funciones los más comunes para la programación modular.

BLOQUE	ALMACENAMIENTO	PROPÓSITO	VALOR RETORNO	USO COMÚN
Bloque Anónimo	No se almacena en BD	Ejecución inmediata	No	Pruebas rápidas, scripts
Procedimiento	Sí (en el DD)	Ejecutar una acción (Hacer algo)	No directo. Utiliza parámetros OUT o IN OUT	Insertar, Actualizar, Borrar, Listados

BLOQUE	ALMACENAMIENTO	PROPÓSITO	VALOR RETORNO	USO COMÚN
Función	Sí (en el DD)	Calcular y devolver un único valor (Obtener algo)	Sí, obligatorio (cláusula RETURN)	Cálculos, validaciones, obtener un dato
Trigger	Sí (en el DD)	Ejecutar automáticamente cuando ocurre un evento (p. ej., un INSERT)	No	Auditoría, mantener integridad

6.1. Parámetros en Procedimientos y Funciones

TIPO	USO	DESCRIPCIÓN	FUNCIÓN	PROCEDIMIENTO
IN	Entrada	El valor llega desde fuera. Es el modo por defecto.	Sí	Sí
OUT	Salida	El valor se genera dentro y es devuelto al bloque que llama.	No	Sí
IN OUT	Entrada/ Salida	El valor es leído al inicio y devuelto al final.	No	Sí

6.2. Invocación

- **Procedimiento:** Se llama como una sentencia PL/SQL independiente:

```
BEGIN
  mi_procedimiento(valor_in, variable_out);
END;
/
```

- **Función:** Se llama como parte de una expresión (se asigna a una variable) o se usa dentro de una sentencia SQL (SELECT):

```
-- En PL/SQL
v_salario_total := calcular_salario_dept(10);
-- En SQL
SELECT nombre, calcular_salario_dept(dept_no) FROM empleados;
```

7. CURSORES

Los cursores son **áreas de memoria** que almacenan datos extraídos de la BD. Se utilizan cuando una consulta puede devolver más de una fila, permitiendo procesar los resultados fila por fila.

7.1. Tipos de Cursores

- **Implícitos:** Declarados automáticamente por Oracle para todas las sentencias DML y para consultas SELECT que devuelven una sola fila.
 - Si devuelven 0 o más de 1 fila, generan las excepciones NO_DATA_FOUND y TOO_MANY_ROWS.
- **Explícitos:** Declarados y nombrados por el programador. Se usan para consultas que devuelven más de una fila.

7.2. Ciclo de Vida del Cursor Explícito

1. **Declaración:** Se define el cursor con una sentencia SELECT (sin INTO).

CURSOR nombre_cursor IS Sentencia_select;

2. **Apertura:** Se ejecuta la consulta y se llena el área de memoria (el cursor).

OPEN nombre_cursor;

3. **Recuperación:** Se leen los datos de la fila actual y se introducen en variables o un registro. Esto se repite en un bucle.

FETCH nombre_cursor INTO var1, var2, ...;

4. **Cierre:** Se desactiva el cursor y se libera la memoria.

CLOSE nombre_cursor;

7.3. Atributos del Cursor Explícito

Se usan para controlar el flujo de la lógica del programa, especialmente en los bucles.

ATRIBUTO	RETORNO	DESCRIPCIÓN
%ISOPEN	Booleano	TRUE si el cursor está abierto.
%NOTFOUND	Booleano	TRUE si la última recuperación (FETCH) no devolvió una fila.
%FOUND	Booleano	TRUE si la última recuperación (FETCH) sí devolvió una fila.
%ROWCOUNT	Número	Número de filas devueltas hasta ese momento.

7.4. Bucle FOR de Cursor

Es una forma simplificada de manejar cursores explícitos: la apertura, recuperación y cierre son implícitos.

FOR nombre_registro IN nombre_cursor LOOP

-- Instrucciones con nombre_registro.columna

END LOOP;

- La variable de registro (nombre_registro) se declara implícitamente.

8. EXCEPCIONES EN PL/SQL

Una excepción es un identificador que surge durante la ejecución provocado por un error. Sirven para capturar y gestionar los errores en tiempo de ejecución.

8.1. Tipos de Excepciones

1. Del **Servidor Oracle (Implícitas):**

- **Con Nombre:** Predefinidas por Oracle (p. ej., NO_DATA_FOUND, TOO_MANY_ROWS, ZERO_DIVIDE).
- **Sin Nombre:** Tienen un código de error (ORA-xxxxx), pero no un nombre predefinido (p. ej., ORA-12560). Se pueden gestionar asignándoles un nombre con PRAGMA EXCEPTION_INIT o usando SQLCODE/SQLERRM.

2. Definidas por el **Usuario (Explícitas)**: Provocadas intencionalmente por el programador (p. ej., para validar lógica de negocio).
 - **Con Nombre**: Se declaran en la sección DECLARE y se provocan con la sentencia **RAISE**.
 - **Sin Nombre**: Se provocan usando el procedimiento **RAISE_APPLICATION_ERROR**.

8.2. Gestión de Excepciones (Manejadores)

Las excepciones se capturan en la sección EXCEPTION del bloque.

- **Sintaxis General**:

```
EXCEPTION
WHEN exception1 [OR exception2...] THEN
    instrucciones; -- Acciones específicas
WHEN OTHERS THEN
    instrucciones; -- Para cualquier otro error no capturado
END;
```

- **Reglas Clave**:
 - Solo se procesa un manejador de excepciones antes de salir del bloque.
 - La cláusula WHEN OTHERS (que captura cualquier error) debe ser la última.
- **Variables de Error**: Para manejar excepciones sin nombre o genéricas, se usan:
 - **SQLCODE**: Devuelve el número del código de error.
 - **SQLERRM**: Devuelve el mensaje asociado al error.